

RK0 User Manual

Using the embedded real-time kernel without surprises

Antonio Giacomelli

Draft for RK0 0.71.0 - 18 August 2026

Contents

Section	Topic	Page
1	About this manual	3
2	Run RK0 on QEMU	4
3	Build-time configuration (kconfig.h)	5
4	A minimal RK0 application	9
5	Tasks and execution progress	10
6	Waiting, timeouts and execution context	12
7	Time services	13
8	Signals and waiting	15
9	Shared-state coordination	17
10	Buffered message passing	19
11	Task-to-task message passing	21
12	Latest-state publication	26
13	Deterministic memory	28
14	Dynamic kernel objects	29
15	Dynamic tasks	32
16	Trace and fault handling	33
17	Choosing a communication service	34
18	Application helper macros	36
19	Compact API index	39
20	Further reading	40

1 About this manual

This manual explains how to use RK0 services and what an application observes. It is deliberately shorter and more operational than the RK0 DocBook.

For each service, the manual asks what relation it expresses, what success means to the caller, and when it should be selected instead of a nearby alternative.

Public prototypes and exhaustive return values remain authoritative in `core/inc/kapi.h`. The DocBook explains the model and design rationale.

This draft describes RK0 0.71.0 at repository commit 6b14715 and DocBook 0710.0.0.0. The principal V0.70.0-V0.71.0 changes are:

- Task roles and the REGULAR/SERVER/RESOURCE_MANAGER split were removed.
- Message Queues and Mailboxes are plain buffered objects with no task owner or owner-priority adoption.
- Direct task messaging now has Synchronous handoff, Invocation, and pool-backed Asynchronous Direct Message contracts.
- A task endpoint may handle Synchronous/Invocation messages or Asynchronous Direct Messages, but not both.
- Selected non-task objects still use bounded dynamic control-block pools when enabled.

1.1 Notation

Priority 0 is the highest user priority. A smaller number means a more urgent task. RUNNING owns the processor; READY may be scheduled; WAITING cannot run until a time or coordination condition is satisfied.

1.2 Return values

Value	Meaning	Examples
0	Operation completed	RK_ERR_SUCCESS
positive	Valid operation; condition not met	queue full, timeout, no waiter
negative	Invalid object, parameter, state or context	null object, wrong owner, ISR misuse

Do not treat every non-zero result as a kernel fault. A non-blocking receive from an empty queue is a normal observation; invalid ownership or context is not.

2 Run RK0 on QEMU

RK0 currently builds with the GNU Arm Embedded toolchain and runs the repository demonstration on QEMU.

Required commands:

```
git clone https://github.com/antoniogiacomelli/RK0.git
cd RK0
make ARCH=armv7m qemu
```

Use `ARCH=armv6m` for the Cortex-M0 QEMU target. The default application is `app/src/application.c`. It contains selectable examples intended as executable documentation.

The principal build outputs are placed below `build/<architecture>/`:

- `rk0_demo.elf` - executable and debugger image;
- `rk0_demo.bin` - raw binary;
- `rk0_demo.hex` - Intel HEX image;
- `rk0_demo.map` - linker map.

For a debug server instead of immediate execution:

```
make ARCH=armv7m qemu-debug
```

QEMU then waits for GDB on TCP port 1234.

3 Build-time configuration (`kconfig.h`)

`core/inc/kconfig.h` defines the kernel instance built into the application. It is included by the common kernel headers, so its values affect kernel sources, public declarations and application code at compile time. Configuration is global: every translation unit must be built against the same file.

Use `ON` and `OFF` for Boolean options. After changing `kconfig.h`, perform a clean rebuild so that no object compiled with the previous feature set remains. Configuration is not a substitute for runtime admission control: pool sizes, task counts, stack sizes and the selected tick must still be justified against the application's memory and timing analysis.

3.1 Clock and kernel tick (`RK_CONF_SYSTICK_DIV`)

`RK_CONF_SYSCORECLK` is the processor clock in hertz. A non-zero value is used directly. On supported hardware builds, zero requests the CMSIS-Core `SystemCoreClock` value; CMSIS-Core is not bundled with RK0, and zero is not valid for the QEMU build.

`RK_CONF_SYSTICK_DIV` is the number of kernel ticks generated per second. Despite its name, it is a frequency divisor applied to the core clock, not the tick duration in milliseconds:

```
systick_counts = RK_CONF_SYSCORECLK / RK_CONF_SYSTICK_DIV;
tick_interval_ms = 1000UL / RK_CONF_SYSTICK_DIV;
```

The Cortex-M port loads `systick_counts - 1` into the 24-bit SysTick reload register. Common values are:

<code>RK_CONF_SYSTICK_DIV</code>	Tick frequency	Kernel tick interval
1000UL	1000 Hz	1 ms
500UL	500 Hz	2 ms
250UL	250 Hz	4 ms
100UL	100 Hz	10 ms

The default is `100UL`, giving a 10 ms tick. A shorter tick improves timeout and release granularity but increases periodic interrupt and timeout-list processing overhead. A longer tick reduces that overhead but quantises more application time values to the same tick.

Choose a tick configuration that satisfies all of these constraints:

- `RK_CONF_SYSTICK_DIV` must be greater than zero and no greater than 1000 for the current integer-millisecond time model;
- `1000UL` should divide evenly by `RK_CONF_SYSTICK_DIV`, otherwise the interval used by the millisecond helpers is truncated;
- the core clock should divide evenly by `RK_CONF_SYSTICK_DIV` when an exact hardware frequency is required;
- `RK_CONF_SYSCORECLK / RK_CONF_SYSTICK_DIV` must be at least 1 and at most `0x1000000`, because Cortex-M SysTick has a 24-bit reload field.

An invalid configuration can leave the computed tick interval at zero and prevents the scheduler from starting. The QEMU unit-test build deliberately overrides `RK_CONF_SYSTICK_DIV` to `100UL`; do not use `RK_QEMU_UNIT_TEST` as an application configuration switch.

3.1.1 Consequences for time APIs

`kTickGet()` returns raw ticks. `kTickGetMs()` and `RK_TICKS_TO_MS()` multiply by the integer tick interval. `RK_MS_TO_TICKS()` divides by it and therefore rounds down:

```
/* With RK_CONF_SYSTICK_DIV == 100UL: one tick is 10 ms. */
RK_TICK a = RK_MS_TO_TICKS(25U); /* 2 ticks: 20 ms */
RK_TICK b = RK_MS_TO_TICKS(5U); /* 0 ticks: RK_NO_WAIT */
```

This is especially important for timeout-bearing calls: a positive millisecond value shorter than one tick becomes `RK_NO_WAIT`. Round up explicitly in application code when the contract is “at least this long”, and check for arithmetic overflow when converting long durations.

Finite time arguments are limited to `RK_MAX_PERIOD`, or `0x7fffffff` ticks. That is approximately 24.9 days with a 1 ms tick and 248.6 days with a 10 ms tick. `RK_WAIT_FOREVER` is a separate sentinel and is not a very long finite timeout.

3.2 Scheduler capacity and system stacks

Symbol	Unit	Effect
<code>RK_CONF_N_USRTASKS_MAX</code>	tasks	Sizes the shared TCB pool for startup and dynamically spawned user tasks. Tasks created by optional facilities through <code>kTaskInit()</code> , such as Trace or an application Logger, also consume a slot.
<code>RK_CONF_IDLE_STACKSIZE</code>	32-bit words	Sizes the Idle task stack, including any application Idle hook.
<code>RK_CONF_POSTPROC_STACKSIZE</code>	32-bit words	Sizes the PostProc stack used for timer callbacks and deferred kernel work.
<code>RK_CONF_DYNAMIC_TASK</code>	ON/OFF	Includes runtime task spawning and termination support.

Both system stack sizes are expressed in words, not bytes. Keep them at or above `RK_MIN_STACKSIZE`, even, and compatible with the port’s 8-byte stack alignment. Stack use must be measured with the actual hooks and callbacks enabled; the defaults are starting values rather than execution-time guarantees.

`RK_CONF_ARMV6M` is normally detected from the compiler architecture macros. It selects smaller default task, trace and dynamic-pool capacities for constrained Cortex-M0-class targets. Override it only when the toolchain does not identify the target correctly.

3.3 Dynamic-object pools

RK_CONF_DYNAMIC_OBJECTS defaults to ON independently of RK_CONF_DYNAMIC_TASK. It enables bounded runtime creation for supported non-task control objects. Set the following maxima to reserve typed kernel pools:

Object family	Maximum-count symbol
Semaphore	RK_CONF_DYNAMIC_SEMAPHORES_MAX
Mutex	RK_CONF_DYNAMIC_MUTEXES_MAX
Sleep Queue	RK_CONF_DYNAMIC_SLEEP_QUEUES_MAX
Message Queue / Mailbox	RK_CONF_DYNAMIC_MESG_QUEUES_MAX
Application Timer	RK_CONF_DYNAMIC_TIMERS_MAX
Most-Recent Message	RK_CONF_DYNAMIC_MRMS_MAX

A maximum is capacity, not a request to instantiate every object. Queue storage, MRM arrays, and direct-message pools remain application-owned. Initialise dynamic control-block pools once with RK_INIT_OBJ_PARTITIONS before the first Create call.

3.4 Service feature gates

Area	Configuration symbols
Time	RK_CONF_CALLOUT_TIMER
Shared state	RK_CONF_SEMAPHORE, RK_CONF_MUTEX, RK_CONF_SLEEP_QUEUE, RK_CONF_CONDVAR
Buffered messages	RK_CONF_MESG_QUEUE, RK_CONF_MESG_QUEUE_SEND_CALLBACK
Direct task messages	RK_CONF_SYNCH_MESG, RK_CONF_ASYNCH_MESG
Latest-state publication	RK_CONF_MRM

Condition-variable helpers require Mutex and Sleep Queue support. Mailboxes use RK_CONF_MESG_QUEUE. RK_CONF_ASYNCH_MESG also requires RK_CONF_MESG_QUEUE because direct-message endpoints use queue infrastructure.

V0.71.0 enables Application Timers by default. Disabling any service remains an architectural decision: rebuild every translation unit against the same kconfig.h.

3.5 Trace configuration

`RK_CONF_TRACE` includes the interactive Trace task and its records. The main sizing and behaviour controls are:

Symbol	Controls
<code>RK_CONF_TRACE_STACKSIZE</code>	Trace task stack in 32-bit words
<code>RK_CONF_TRACE_PRIO</code>	Trace task priority
<code>RK_CONF_TRACE_MAX_OBJECTS</code>	Number of registered objects Trace can retain
<code>RK_CONF_TRACE_LINE_LEN</code>	Interactive command-line buffer length
<code>RK_CONF_TRACE_RECORD_DEPTH</code>	Per-object history depth
<code>RK_CONF_TRACE_OVERFLOW_BACKLOG</code>	Retained records after the normal history overflows
<code>RK_CONF_TRACE_FRAME_BUFFER_DEPTH</code>	Buffered machine-readable trace frames
<code>RK_CONF_TRACE_FRAME_STDOUT</code>	Immediate frame output instead of buffering
<code>RK_CONF_TRACE_TASK_PRIO_HISTORY</code>	Recording of task effective-priority changes

Trace storage grows with the configured depths and object/task capacities. Trace timing also depends on its priority and output backend. Size it as part of the target application, and do not assume that a development Trace configuration has zero interference.

3.6 Debug/release error checking (`NDEBUG`) and test overrides

In a build without `NDEBUG`, `kconfig.h` enables `RK_CONF_ERR_CHECK`, `RK_CONF_FAULT` and `RK_CONF_FAULT_PRINT_STDERR`. These checks detect invalid objects, parameters, contexts and protocol use; they do not turn well-defined unsuccessful results such as timeout or queue-full into faults.

Defining `NDEBUG` removes this default checking configuration. API preconditions then become application obligations, and invalid use may no longer produce the diagnostic return or halt observed in a debug build. Validate the release configuration independently rather than treating `NDEBUG` as only an assertion switch.

`RK_QEMU_UNIT_TEST` overrides the user-task capacity and tick configuration required by the repository tests. This block is test infrastructure and should not be edited to tune an application.

4 A minimal RK0 application

An RK0 application provides ordinary platform initialisation, static kernel storage and a mandatory `kApplicationInit()` function.

```
#include <kapi.h>

#define WORKER_STACK_WORDS 160U
#define WORKER_PRIORITY    2U

RK_DECLARE_TASK(workerHandle, WorkerTask,
                workerStack, WORKER_STACK_WORDS)

int main(void)
{
    kCoreInit();
    kInit();

    /* kInit() does not return during normal operation. */
    while (1)
    {
        kErrorHandler(RK_FAULT_APP_CRASH);
    }
}

VOID kApplicationInit(VOID)
{
    /* Required before any dynamic-object Create operation. */
    RK_INIT_OBJ_PARTITIONS

    RK_ERR err = kCreateTask(&workerHandle,
                            WorkerTask,
                            RK_NO_ARGS,
                            "Worker",
                            workerStack,
                            WORKER_STACK_WORDS,
                            WORKER_PRIORITY,
                            RK_PREEMPT);

    K_ASSERT(err == RK_ERR_SUCCESS);
}

VOID WorkerTask(VOID *args)
{
    RK_UNUSEARGS

    while (1)
    {
        DoOneCycle();
        kSleepPeriodic(RK_MS_TO_TICKS(20U));
    }
}
```

`RK_DECLARE_TASK` declares the entry prototype, an 8-byte-aligned stack and a task handle. Stack size is expressed in 32-bit words, not bytes. The handle and stack must remain valid for the application lifetime; file-scope storage is normal.

`kInit()` calls `kApplicationInit()`, completes kernel initialisation, makes the created tasks ready, enables interrupts and starts the highest-priority task. Create startup tasks and initialise the objects they need inside `kApplicationInit()`.

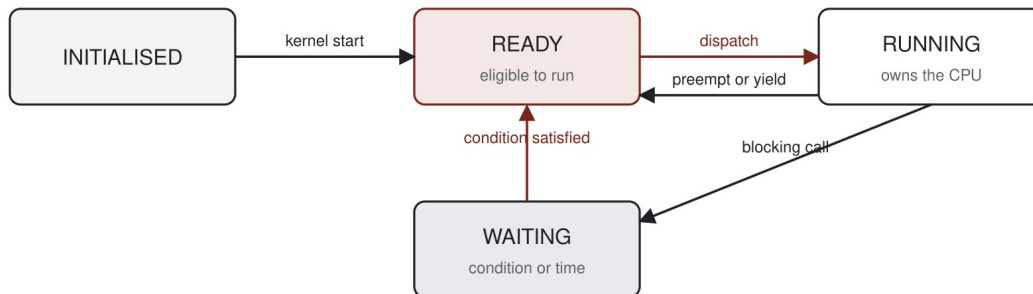
`RK_INIT_OBJ_PARTITIONS` initialises the fixed pools used by runtime-created kernel objects. It is a no-op when `RK_CONF_DYNAMIC_OBJECTS` is disabled, so it may remain in common startup code. Call it once before any dynamic-object `Create` operation. It does not create an object.

5 Tasks and execution progress

5.1 Task states

The scheduler is the only component that changes a task from READY to RUNNING.

Only the scheduler changes READY to RUNNING



RK0 task lifecycle

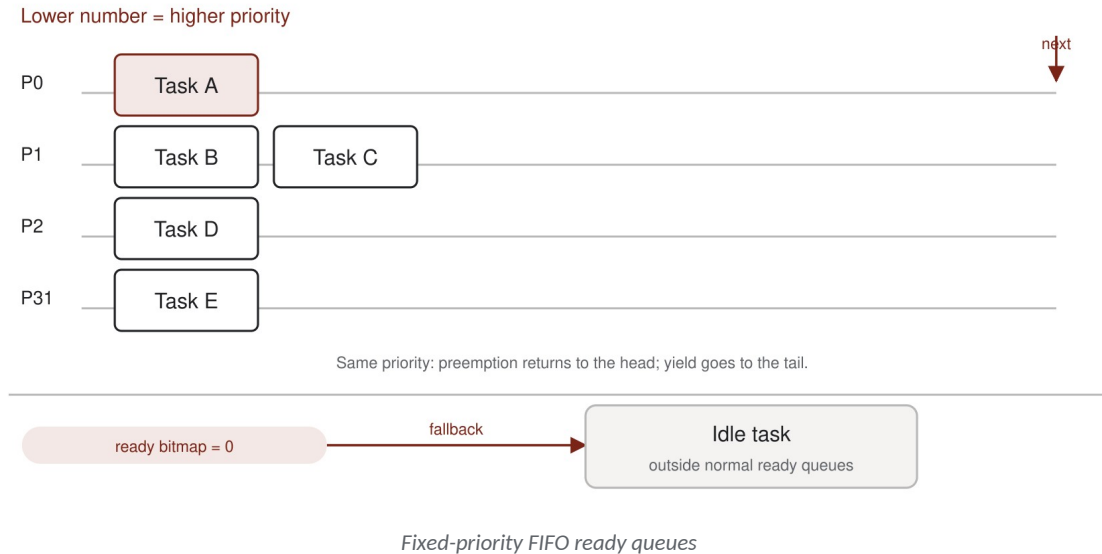
A static task normally moves among INITIALIZED, READY, RUNNING and a family of WAITING states. A dynamic task may additionally become TERMINATED.

- A task becomes **READY** when creation completes after scheduler start, or when its waiting condition is satisfied.
- The scheduler changes one READY task to **RUNNING**.
- A higher-priority ready task may preempt a preemptible running task.
- `kYield()` voluntarily returns the caller to READY.
- A blocking service changes the caller to a service-specific WAITING state.

Returning from a task entry function is not a task-termination operation. Task functions should normally contain a loop. Dynamic tasks terminate through the dynamic-task API described later.

5.2 Fixed-priority FIFO scheduling

RK0 maintains one FIFO ready queue for each normal priority from 0 to 31 and does not perform automatic time slicing. A 32-bit bitmap records which of those queues are non-empty.



When the bitmap is non-zero, its least-significant set bit identifies the highest-priority non-empty ready queue. Within the same priority:

- initial order follows task creation order;
- a preempted task returns to the head, so it resumes before equal-priority peers;
- a yielded task goes to the tail;
- a task made ready after waiting goes to the tail.

The Idle task is not on the normal ready queues and is outside their bitmap. It is dispatched as the fallback when the ready bitmap is zero; it is not the priority-31 task and it does not compete with application work.

Two equal-priority tasks therefore cooperate by yielding or waiting. If the running task does neither, there is no expressed reason for another equal-priority task to run.

`RK_NO_PREEMPT` creates an exceptional non-preemptible task. Once dispatched, it continues until it blocks, yields or otherwise leaves RUNNING. Use this only for short, bounded service routines: it can delay every user task, including more urgent ones.

5.3 Yielding is not sleeping

`kYield()` says that the task has completed its present turn among ready peers. It does not express a time condition and does not make the task WAITING.

If no equal- or higher-priority task is ready, yielding does not manufacture work for a lower-priority task. The caller remains the correct task to run.

5.4 Scheduler lock

`kSchLock()` and `kSchUnlock()` (also exposed as `kPreemptDisable()` and `kPreemptEnable()`) defer user-task preemption. Locks are nested and belong to the task.

The scheduler lock does **not** disable interrupts and is not a mutex. It does not establish shared-resource ownership or priority inheritance. Use it only where delaying dispatch is itself the required, bounded operation.

6 Waiting, timeouts and execution context

Services that may wait accept one of three timeout forms:

Timeout	Caller behaviour
RK_NO_WAIT	Inspect or attempt the operation and return immediately
finite ticks	Wait up to the requested kernel time
RK_WAIT_FOREVER	Wait until the service condition is satisfied

Finite values must not exceed `RK_MAX_PERIOD`. `RK_MS_TO_TICKS()` truncates, so a duration shorter than one tick becomes `RK_NO_WAIT` for timeout-bearing calls.

6.1 What a timeout guarantees

A timeout removes a task's dependency and makes it `READY`. Actual return still depends on fixed-priority scheduling. The return code identifies which completion occurred.

For Synchronous Invocation, a timeout before server acceptance removes the pending call. A timeout after acceptance abandons the reply from the caller's perspective; the server still closes the accepted rendezvous with `kSynchMesgReply()`, but no reply is copied.

6.2 Interrupt service routines

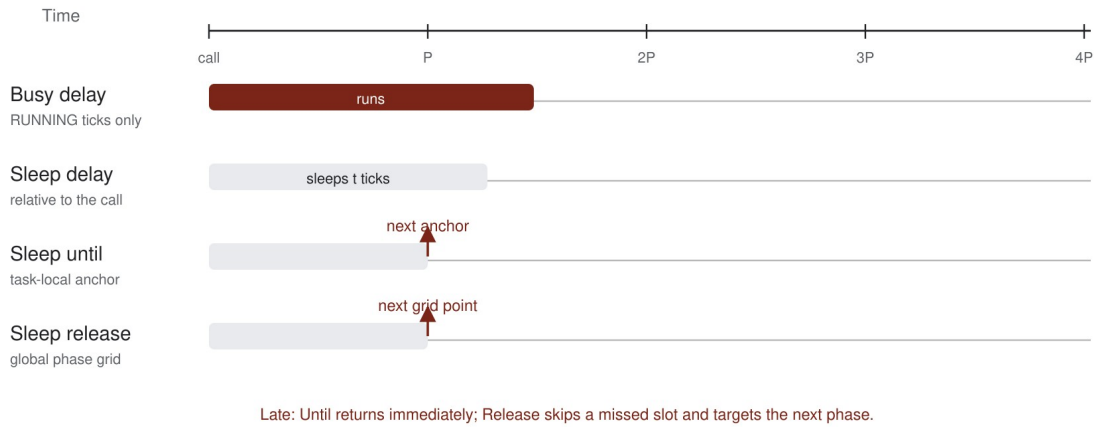
An ISR must never request a blocking operation. Some services expose non-blocking ISR paths, such as `kEventSet()`, `kSemaphorePost()`, and documented no-wait Message Queue calls. Multi-task wake work may be deferred to `PostProc`.

Synchronous/Invocation and Asynchronous Direct Message endpoints are task-context services. `kMesgSend()` is non-blocking, but it is still rejected from ISR context. Check `kapi.h` for the exact allowed context rather than inferring ISR safety from the absence of a timeout parameter.

Asynchronous Direct Message send has no timeout: on success ownership moves immediately to the receiver endpoint. Pool exhaustion is handled at allocation time by application policy.

7 Time services

RK0 keeps different meanings of time in separate calls.



Comparison of RK0 time services

7.1 Choose the intended time relation

Service	Meaning	Best use
<code>kBusyDelay(t)</code>	Consume <code>t</code> ticks while RUNNING	Simulated or deliberately active workload
<code>kSleepDelay(t)</code>	Sleep relative to this call	One-off delay or back-off
<code>kSleepUntil(&anchor, p)</code>	Advance one task-local deadline by <code>p</code>	Periodic work that must account for every activation
<code>kSleepRelease(p)</code>	Target the next scheduler-start phase slot	Phase-aligned periodic work that may skip missed releases

7.2 Busy Delay

`kBusyDelay(t)` leaves the caller RUNNING. Only ticks observed while the caller executes count towards completion. Time spent preempted does not reduce the remaining busy delay.

Use it to represent CPU work in an example or for a genuinely active wait. It does not free the processor for a lower-priority task.

7.3 Sleep Delay

`kSleepDelay(t)`, also named `kSleep(t)`, suspends the task for a relative duration measured from the call.

```
DoWork();
kSleep(RK_MS_T0_TICKS(50U));
```

In a loop, work time, preemption and release jitter accumulate. Do not use relative sleep to define a periodic task.

7.4 Sleep Until

`kSleepUntil()` uses a reference owned by the task:

```
RK_TICK anchor = kTickGet();
while (1)
{
    AcquireAndProcessSample();
    RK_ERR err = kSleepUntil(&anchor,
                           RK_MS_TO_TICKS(10U));
    if (err == RK_ERR_ELAPSED_PERIOD)
    {
        RecordLateActivation();
    }
}
```

Each call advances the anchor by one period. If the deadline has already elapsed, the call returns immediately with `RK_ERR_ELAPSED_PERIOD`. It does not skip forward over multiple releases. This favours execution count over absolute phase recovery.

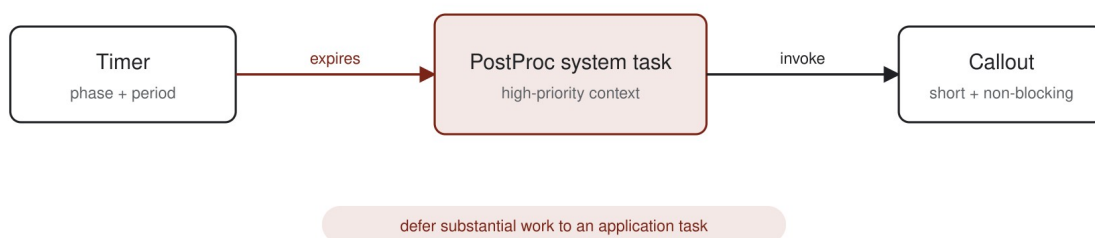
7.5 Sleep Release

`kSleepRelease()`, also named `kSleepPeriodic()`, uses the phase grid established when scheduling begins. A late task sleeps less to reach the next phase. If an entire release has been missed, RK0 records an overrun, skips the missed slot and targets the next valid one.

Use it for known startup tasks that should remain aligned to a system phase. Do not use the global startup phase of `kSleepRelease()` for a task spawned later at an arbitrary time; use a task-local `kSleepUntil()` anchor instead.

7.6 Application timer callouts

An application timer defers a callback from tick expiry to the PostProc system task.



Application timer callout context

`kTimerInit()` arms a one-shot or reloading timer with an initial phase, a period, a callback and an argument pointer. `kTimerCancel()` cancels an active timer. When dynamic objects are enabled, `kTimerCreate()` allocates and arms a timer, and `kTimerDestroy()` cancels and releases it.

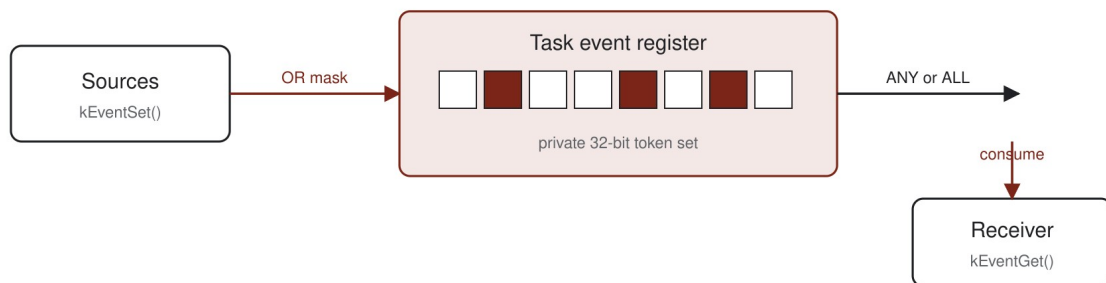
Callouts execute in high-priority, non-preemptible system-task context. They must be short and non-blocking. A normal pattern is to set a task event or post a semaphore and let an application task perform the work.

8 Signals and waiting

Signals express occurrence or availability. They do not carry an application payload.

8.1 Task Event Register

Every task has a private 32-bit Event Register. Other tasks or an ISR set bits; only the receiving task waits for and consumes them.



Repeated sets of the same bit collapse into one pending token.

Task Event Register

`kEventSet(task, mask)` ORs the supplied mask into the current register. Setting the same bit repeatedly before it is consumed still leaves one bit set. If each occurrence matters, use a counting Semaphore or a Message Queue.

The receiver selects `RK_EVENT_ANY` or `RK_EVENT_ALL`:

```
#define EVT_SAMPLE    RK_EVENT_1
#define EVT_SETPoint RK_EVENT_2

RK_TASK_EVENT got = 0UL;

RK_ERR err = kEventGet(EVT_SAMPLE | EVT_SETPoint,
                      RK_EVENT_ALL,
                      &got,
                      RK_WAIT_FOREVER);
K_ASSERT(err == RK_ERR_SUCCESS);
```

On success, RK0 optionally copies the complete register state to `got`, then clears the requested mask. Bits outside that mask remain pending.

Use Task Events when:

- the receiver is known;
- no payload is needed;
- bitwise ANY or ALL composition is useful;
- repeated occurrences may coalesce.

`kEventQuery()` observes the current register. `kEventClear()` explicitly clears selected bits. Passing `NULL` as their task handle selects the calling task; an ISR must name a target task explicitly.

8.2 Semaphores

A Semaphore stores an unsigned count of available units or pending occurrences. It has no owner.

```
static RK_SEMAPHORE samplesReady;

VOID kApplicationInit(VOID)
{
    RK_ERR err = kSemaphoreInit(&samplesReady,
                                0U, /* initially no sample */
                                8U); /* retain up to 8 */
    K_ASSERT(err == RK_ERR_SUCCESS);
}
```

`kSemaphorePend()` consumes a stored count. If the count is zero, it returns immediately for `RK_NO_WAIT` or places the task in the priority-ordered wait queue.

`kSemaphorePost()` first satisfies the most urgent waiting task. With no waiter, it increments the count up to the configured maximum. Posting at the maximum leaves the value unchanged and returns `RK_ERR_SEMA_FULL`.

Convenience initialisers are available:

```
kSemaBinInit(&doorbell, 0U); /* maximum 1 */
kSemaCountInit(&credits, 12U); /* maximum UINT */
```

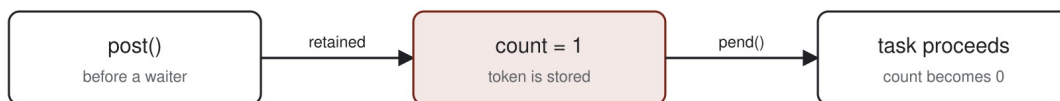
A binary Semaphore can guard a simple region only by application convention. It is not a Mutex: any task may post it, there is no owner and priority inversion is not handled.

Dynamic use is `RK_SEMAPHORE_HANDLE` with `kSemaphoreCreate()` and `kSemaphoreDestroy()`. Destroy requires no waiters; a stored count is discarded.

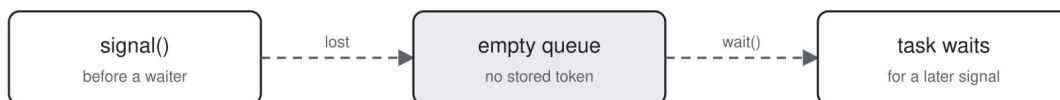
8.3 Sleep Queues

A Sleep Queue is a pure, non-latching waiting relation. It stores waiting tasks, not a condition and not an occurrence token.

Semaphore



Sleep Queue



Semaphore token retention compared with a Sleep Queue

`kSleepQueueSleep()` waits for a future signal. `kSleepQueueSignal()` wakes the highest-priority waiter; `kSleepQueueWake(q, n, left)` wakes up to `n` (0 means all). `kSleepQueueReady()` wakes a named task from that queue, while `kSleepQueueUnready()` moves a non-running READY task into it.

9 Shared-state coordination

9.1 Mutex Locks

A Mutex expresses exclusive ownership of shared state. Once locked, only its owner may unlock it. RK0 Mutexes are non-recursive: attempting to lock the same Mutex twice is invalid.

```
static RK_MUTEX stateLock;

VOID kApplicationInit(VOID)
{
    RK_ERR err = kMutexInit(&stateLock,
                           RK_PRIO_INHERITANCE);
    K_ASSERT(err == RK_ERR_SUCCESS);
}

VOID UpdateState(VOID)
{
    RK_ERR err = kMutexLock(&stateLock, RK_WAIT_FOREVER);
    K_ASSERT(err == RK_ERR_SUCCESS);

    sharedState.value++;

    err = kMutexUnlock(&stateLock);
    K_ASSERT(err == RK_ERR_SUCCESS);
}
```

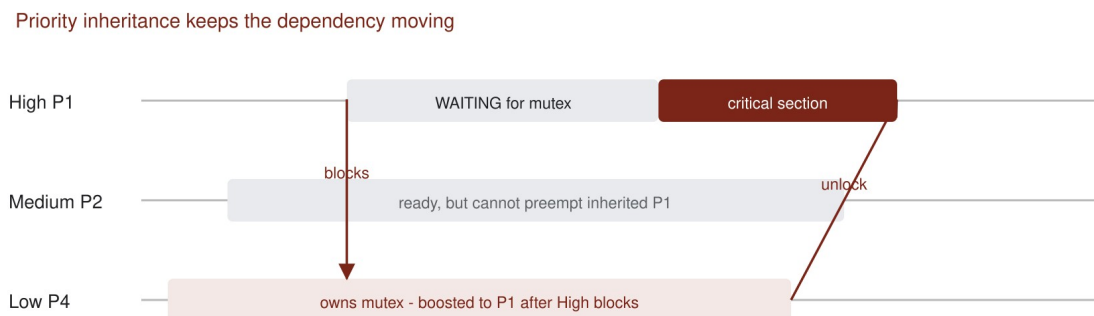
The dynamic form uses `RK_MUTEX_HANDLE`, `kMutexCreate()` and `kMutexDestroy()`. The Mutex must be unlocked, unowned and have no waiters before it can be destroyed.

The protocol is selected at initialisation:

- `RK_PRIO_NONE` - mutual exclusion only;
- `RK_PRIO_INHERITANCE` - transitive priority inheritance.

9.1.1 Priority inheritance

When a high-priority task waits for a Mutex owned by a lower-priority task, the owner inherits the waiter's effective priority. This prevents unrelated medium-priority work from extending the inversion.



Transitive priority inheritance on a Mutex

RK0 recomputes a task's effective priority across all PI-enabled Mutexes it owns. If an owner itself waits on another Mutex, the inherited priority propagates through the dependency chain.

Priority inheritance bounds interference; it does not make long critical sections inexpensive. Keep the protected region bounded and do not perform unrelated blocking operations while owning a Mutex.

9.1.2 Mutex versus binary Semaphore

Property	Mutex	Binary Semaphore
Stored state	locked/unlocked	count 0/1
Owner	yes	no
Who may release	owner only	any task or ISR permitted by API
Priority inheritance	optional	no
Intended meaning	exclusive shared-state access	occurrence or availability

9.2 Condition-variable helpers

RK0 pairs a Sleep Queue with a PI-enabled Mutex. Initialise both once with `kCondVarInit()`:

```
while (!PredicateIsTrue())
{
    RK_ERR err = kCondVarWait(&condition,
                             &stateLock,
                             RK_WAIT_FOREVER);
    K_ASSERT(err == RK_ERR_SUCCESS);
}
```

The caller tests an application predicate while owning the Mutex. Waiting releases the Mutex and joins the Sleep Queue atomically. If the Mutex is released, the helper attempts to reacquire it before returning, including after timeout or no-wait; the timeout bounds the complete wait-plus-lock operation.

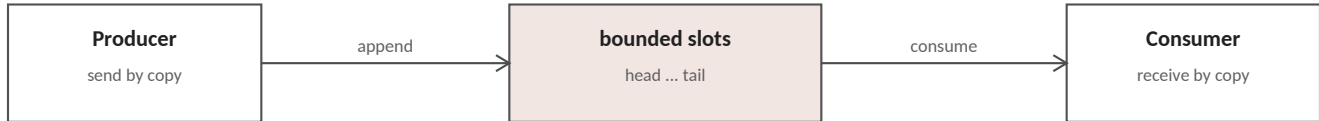
Always retest the predicate in a loop. A wake means that the condition may have changed; it is not proof that this particular task may proceed.

`kCondVarSignal()` wakes one waiter. `kCondVarBroadcast()` wakes all. The application decides which signalling discipline preserves its invariant.

10 Buffered message passing

10.1 Message Queues

A Message Queue transfers fixed-size messages by copy and preserves their order up to the configured capacity. Queues and Mailboxes are plain buffered objects: they have no task owner and do not adopt a receiver priority.



A successful send means the message was copied directly to an already-waiting receiver or admitted to queue storage. It does not mean the receiver processed the message.

10.1.1 Declare and initialise

```

typedef struct
{
    ULONG timestamp;
    UINT value;
} Sample_t;

#define SAMPLE_DEPTH 4U
RK_DECLARE_MESG_QUEUE(sampleQueue, sampleQueueStorage,
                      Sample_t, SAMPLE_DEPTH)

RK_ERR err = kMsgQueueInit(&sampleQueue,
                          sampleQueueStorage,
                          RK_MESGQ_MESG_SIZE(Sample_t),
                          SAMPLE_DEPTH);
K_ASSERT(err == RK_ERR_SUCCESS);
  
```

Queue slots are 1, 2, 4, or 8 ULONG words. Larger payloads should use a fixed-block Memory Partition and a pointer queue, or the V0.71.0 Asynchronous Direct Message service when task-addressed ownership transfer is the intended contract.

Capacity absorbs bursts and jitter; it does not create throughput. Over time the producer waits, the application drops messages deliberately, or the consumer sustains the arrival rate.

10.1.2 Send and receive

```
Sample_t sample = ReadSample();
RK_ERR err = kMsgQueueSend(&sampleQueue, &sample, RK_NO_WAIT);
if (err == RK_ERR_BUFFER_FULL)
{
    RecordDroppedSample();
}
```

kMsgQueueSend() appends; kMsgQueueJam() inserts at the front. A sender may wait on a full queue and a receiver may wait on an empty queue. Waiters are selected by effective priority, FIFO among equals.

- kMsgQueuePeek() copies the front message without consuming it.
- kMsgQueueQuery() reports buffered messages and waiting senders/receivers.
- kMsgQueueReset() discards buffered messages and releases current waiters.
- kMsgQueueDestroy() requires an empty queue with no waiting or broadcast receivers; there is no owner state.

10.1.3 Notify callback

When enabled, kMsgQueueInstallSendCbK() runs a short non-blocking callback after a successful normal send or jam. The callback is notification, not delivery; the queue remains the message authority.

10.2 Mailboxes

RK_MBOX is an alias for a Message Queue of depth one. Ordinary post and pend retain queue semantics. kMsgQueuePostOvw() replaces the pending value; kMboxBroadcast() hands one copied value to tasks already blocked in kMboxBroadcastRecv().

Operation	Admission condition	Retained result
ordinary post	slot empty or receiver waiting	one queued value
overwrite	any mailbox state	newest pending value
broadcast	broadcast receivers already waiting	copy delivered to all targeted waiters

Mailbox broadcast is a simultaneous handoff to present waiters. It does not retain publication for future readers; use MRM for latest-state publication to many readers.

10.2.1 One-slot behaviours

Mailbox form	Use when
Overwrite	Only the newest not-yet-consumed value matters to one receiver
Broadcast	One value is copied to every task already blocked as a broadcast receiver

Overwrite is not MRM: normal mailbox receivers compete for one slot. Broadcast is not history: with no waiting broadcast receiver it deposits nothing and returns `RK_ERR_BUFFER_EMPTY`.

11 Task-to-task message passing

Direct messages name a task handle. Endpoint state lives in the receiver TCB rather than in a standalone queue object. V0.71.0 exposes three distinct completion contracts:

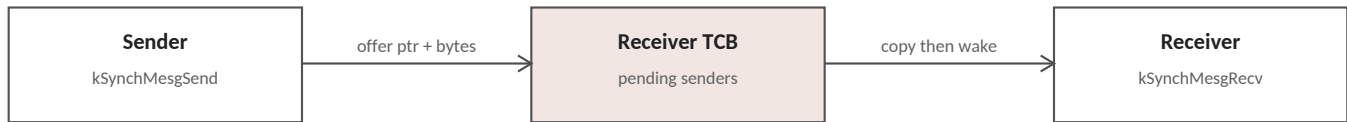
Contract	Successful sender/caller means	Receiver operation
Synchronous	receiver copied the offered payload	<code>kSynchMesgRecv()</code>
Invocation	server copied request and replied	<code>kSynchMesgAccept()/Reply()</code>
Asynchronous Direct	receiver endpoint owns the <code>RK_MESG</code>	<code>kMesgWait()</code>

A task may initialise a Synchronous endpoint (which also supports Invocation) or an Asynchronous Direct Message endpoint, but not both. This gives each task one unambiguous receive policy.

V0.70.0 removed task roles. Any ordinary task may be used as a receiver or server when its endpoint and application protocol require it.

11.1 Synchronous Message (rendezvous)

The sender offers one bounded payload and blocks until the named receiver copies it into receiver-owned storage. No message is deposited in kernel payload storage.



Success means the receiver copied the payload. It does not mean that the receiver processed it or produced a reply. Multiple senders may wait and are selected by effective priority.

```

/* Startup: configure the receiver endpoint maximum. */
kSynchMesgInit(controllerHandle, sizeof(Sample_t));

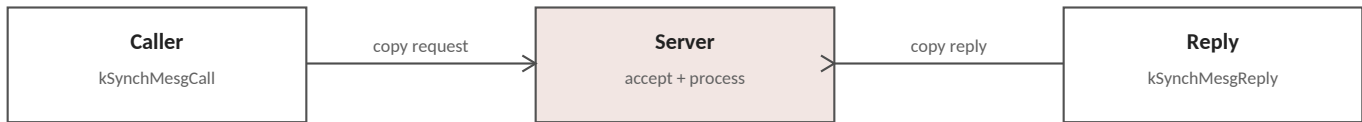
/* Sender: source remains valid until the call returns. */
kSynchMesgSend(controllerHandle, &sample,
               sizeof(sample), RK_WAIT_FOREVER);

/* Receiver: destination holds the copied payload. */
Sample_t received;
ULONG copiedBytes = 0UL;
kSynchMesgRecv(&received, &copiedBytes, RK_WAIT_FOREVER);
  
```

- The endpoint maximum is non-zero and a multiple of RK_WORD_SIZE; each handoff supplies its actual byte count.
- If a sender times out, its source pointer is invalidated for the receiver; a later receive cannot consume stale data.
- A high-priority blocked sender contributes priority to a lower-priority receiver until the waiting relation changes.
- Send and receive reject a running task that owns a Mutex, preserving one coordination authority.

11.2 Invocation (extended rendezvous)

Invocation uses the same Synchronous endpoint but extends request copy with server acceptance, processing, and a reply copied back to the caller.



kSynchMesgSend() completes on receive copy. kSynchMesgCall() (alias kSynchMesgInvoke()) completes only when the server replies or the caller timeout expires.

```

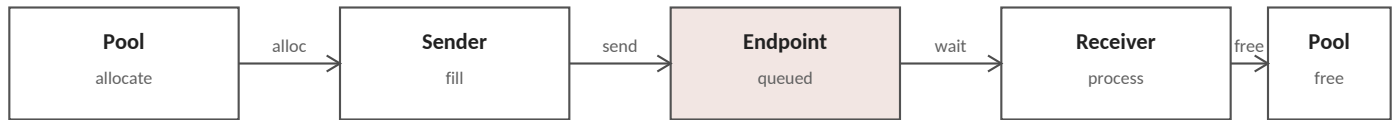
Request_t req = {1U, value};
Reply_t reply = {RK_ERR_SUCCESS, 0UL};
ULONG replyBytes = 0UL;
RK_SYNCH_ATTR attr = {
    &req, sizeof(req), &reply, sizeof(reply), &replyBytes
};
kSynchMesgCall(serverHandle, &attr, RK_WAIT_FOREVER);

RK_SYNCH_CALL_DATA call = {0};
Request_t accepted;
ULONG reqBytes = 0UL;
kSynchMesgAccept(&call, &accepted, &reqBytes, RK_WAIT_FOREVER);
Reply_t result = {RK_ERR_SUCCESS, accepted.value + 1UL};
kSynchMesgReply(&call, &result, sizeof(result));
  
```

- The request pointer remains valid until accept; the caller-owned reply buffer remains valid until call returns.
- A timeout before accept removes the pending caller. After accept, the caller abandons the reply but the server still closes the active call.
- The server receives the active caller's priority contribution until reply or abandoned-call completion.
- RK_NO_WAIT is invalid for kSynchMesgCall(); use bounded ticks or RK_WAIT_FOREVER.

11.3 Asynchronous Direct Message

An RK_MESG is allocated from an application-provided fixed-block pool. kMesgSend() transfers ownership to a named task endpoint without waiting for the receiver to copy or process it.



Successful send means the receiver endpoint owns the message object. The sender must not touch it again. If the receiver is waiting, it is readied with the pointer; otherwise the message remains queued at the endpoint.

```

typedef struct
{
    UINT source;
    UINT sequence;
    ULONG value;
} AsyncPayload_t;

RK_DECLARE_MESG_POOL(asyncPool, asyncPoolBuf,
    AsyncPayload_t, 4U)

kMesgPoolInit(&asyncPool, asyncPoolBuf,
    sizeof(AsyncPayload_t), 4U);
kMesgEndpointInit(receiverHandle);
  
```

- kMesgAlloc() returns NULL when the selected pool has no free block; allocation policy remains in the application.
- The payload is accessed through kMesgPayload() or RK_MESG_PAYLOAD(message, Type).
- kMesgWait(RK_ANY_TASK, ...) accepts any sender; a specific task handle filters without discarding other queued messages.
- kMesgFree() returns a received or unsent allocated message to its originating pool.

RK_CONF_ASYNC_MESG requires RK_CONF_MESG_QUEUE. Endpoint queues are task-backed; the message pool itself is an application-owned Memory Partition.

11.3.1 Usage example

```

RK_MSG *msgPtr = kMsgAlloc(&asyncPool);
K_ASSERT(msgPtr != NULL);

AsyncPayload_t *payloadPtr =
    RK_MSG_PAYLOAD(msgPtr, AsyncPayload_t);
payloadPtr->sequence = ++sequence;
payloadPtr->value = ReadValue();

RK_ERR err = kMsgSend(receiverHandle, msgPtr);
K_ASSERT(err == RK_ERR_SUCCESS);
/* Ownership moved: sender no longer touches msgPtr. */

```

```

RK_MSG *msgPtr = NULL;
RK_ERR err = kMsgWait(RK_ANY_TASK, &msgPtr,
                     RK_WAIT_FOREVER);
K_ASSERT(err == RK_ERR_SUCCESS);

AsyncPayload_t *payloadPtr =
    RK_MSG_PAYLOAD(msgPtr, AsyncPayload_t);
Consume(payloadPtr->value);

err = kMsgFree(msgPtr);
K_ASSERT(err == RK_ERR_SUCCESS);

```

11.4 Choose by completion and ownership

Contract	Sender-side lifetime	Success means	Retained history
Synchronous	sender retains source while blocked	receiver copied payload	none
Invocation	caller retains request/reply buffers	server replied	none
Asynchronous Direct	ownership moves to endpoint	endpoint owns RK_MSG	queued messages

Use Synchronous handoff when delivery copy is the completion boundary. Use Invocation when a server result is required. Use Asynchronous Direct Message when the sender must continue immediately and explicit message-object ownership is acceptable.

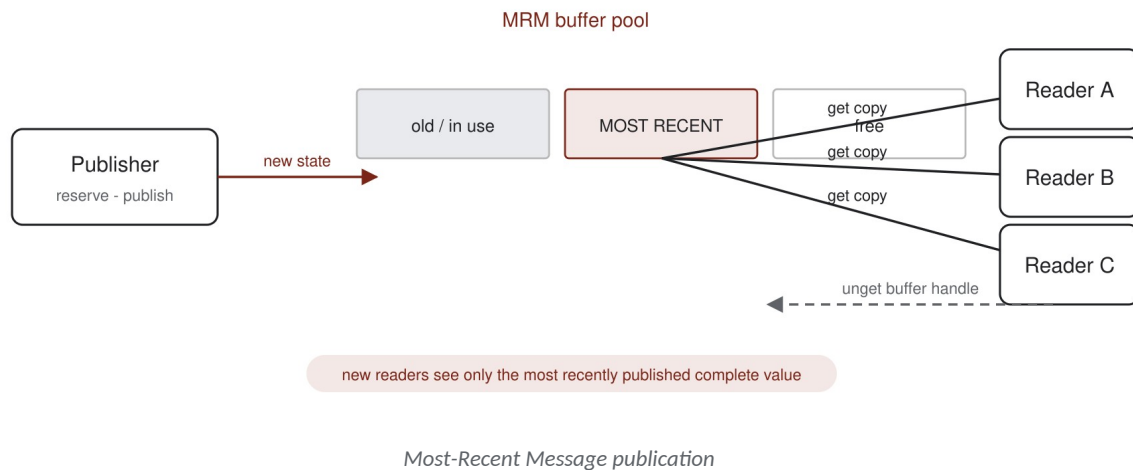
11.5 Endpoint and lifetime rules

- Synchronous/Invocation and Asynchronous endpoints are mutually exclusive per receiver task.
- Pool storage and any message not yet transferred remain application-owned; a successful async send transfers the block.
- Sender identity is recorded in RK_MSG and available through kMsgGetSenderHandle() or kMsgGetSenderID().
- Direct endpoints are task-context services and appear in Trace list kipc state.

12 Latest-state publication

12.1 Most-Recent Message

MRM is a non-blocking 1:N protocol for publishing the latest complete state. New readers never walk through a backlog of obsolete samples.



Use it when:

- one writer publishes state;
- several readers operate at independent rates;
- the latest complete value is more relevant than every intermediate value;
- readers must never observe a partially updated object.

12.1.1 Configure both pools

MRM uses two equal-length pools:

1. `RK_MRM_BUF` control buffers, which hold reader counts and payload addresses;
2. application payload buffers.

For one writer and R readers, configure $R + 2$ elements: one per communicating task plus one spare. For an application type `PlantState`:

```
#define N_READERS 3U
#define N_MRM_BUFS (N_READERS + 2U)

static RK_MRM stateMrm;
static RK_MRM_BUF stateControlPool[N_MRM_BUFS];
static PlantState statePayloadPool[N_MRM_BUFS] K_ALIGN(4);

RK_ERR err = kMRMInit(&stateMrm, stateControlPool,
                    statePayloadPool, N_MRM_BUFS,
                    RK_TYPE_WORD_COUNT(PlantState));
K_ASSERT(err == RK_ERR_SUCCESS);
```

Use static zero-initialised storage for the `RK_MRM_BUF` pool.

`kMRMCreate()` allocates only the MRM control object. The control-buffer and payload pools remain application-owned. Before `kMRMDestroy()`, the application must ensure that no reader retains a version and

no writer reservation remains outstanding. An unreferenced current publication is reclaimed during destruction.

12.1.2 Writer: reserve and publish

```
RK_MRM_BUF *buffer = kMRMReserve(&stateMrm);
if (buffer != NULL)
{
    PlantState state = ReadPlantState();
    RK_ERR err = kMRMPublish(&stateMrm,
                            buffer,
                            &state);
    K_ASSERT(err == RK_ERR_SUCCESS);
}
```

Reservation is private to the writer until publication. Publishing copies the payload and makes that buffer the only version visible to new readers.

12.1.3 Reader: get and unget

```
PlantState state;
RK_MRM_BUF *buffer = kMRMGet(&stateMrm, &state);

if (buffer != NULL)
{
    ProcessState(&state);

    RK_ERR err = kMRMUnget(&stateMrm, buffer);
    K_ASSERT(err == RK_ERR_SUCCESS);
}
```

`kMRMGet()` copies the current payload and returns a handle to the version from which it was taken. Pair every successful get with exactly one unget of that same handle. The handle lets RK0 retain an old version while readers that acquired it are still active.

MRM never waits, wakes a task or donates priority. `kMRMReserve()` returns `NULL` when no buffer is available; `kMRMGet()` returns `NULL` before the first publication.

13 Deterministic memory

13.1 Memory Partitions

A Memory Partition allocates homogeneous fixed-size blocks from application-supplied storage.

Fixed-size block pool



No variable-size search, coalescing, or heap fragmentation model.

Fixed-block Memory Partition

Allocation and free are immediate; they do not wait or affect task priority. There is no variable-size search, block splitting, coalescing or general heap-fragmentation model.

```
typedef struct
{
    RK_TICK timestamp;
    CHAR text[60];
} LogRecord;

#define LOG_BLOCKS 8U

static RK_MEM_PARTITION logMemory;
RK_DECLARE_MEM_POOL(LogRecord, logPool, LOG_BLOCKS)

VOID InitLogMemory(VOID)
{
    RK_ERR err = kMemPartitionInit(&logMemory,
                                   logPool,
                                   sizeof(LogRecord),
                                   LOG_BLOCKS);
    K_ASSERT(err == RK_ERR_SUCCESS);
}
```

Allocate and test for exhaustion:

```
LogRecord *record = kMemPartitionAlloc(&logMemory);
if (record != NULL)
{
    FillRecord(record);
}
```

Return the block exactly once to its original partition:

```
kMemPartitionFree(&logMemory, record);
```

Block size is rounded up to a word boundary. The pool must be aligned, sized for the rounded block geometry and contain at least one block. Allocation does not clear a reused block; initialise application fields before use.

13.1.1 Mail queue pattern

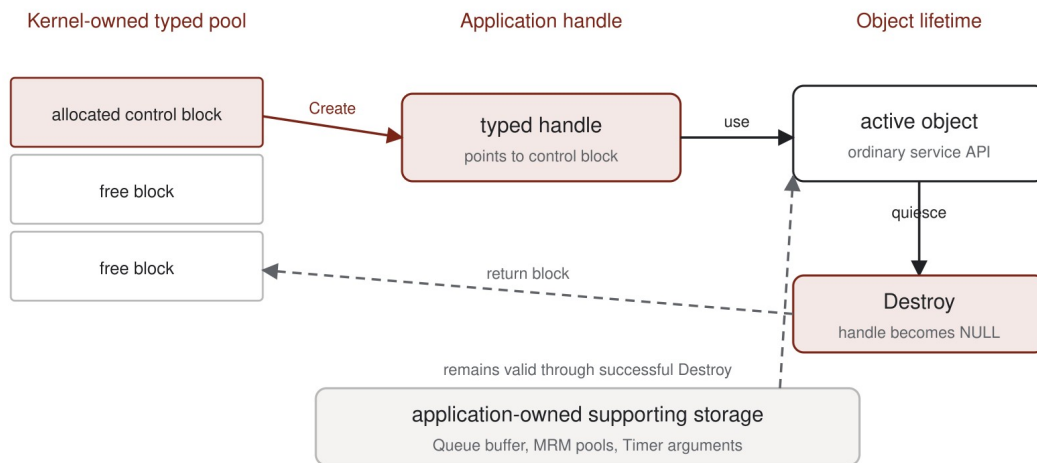
For variable or larger payloads, combine a Memory Partition with a one-word Message Queue of pointers:

1. sender allocates a block;
2. sender fills it and enqueues its pointer;
3. receiver dequeues and processes the block;
4. receiver returns it to the same partition.

This keeps kernel copy cost bounded to one word while making lifetime and ownership explicit. The sender must free the block if a non-blocking queue send fails.

14 Dynamic kernel objects

Dynamic creation changes an object's storage and lifetime; it does not change the service's completion, waiting or priority semantics. RK0 still uses fixed-capacity, kernel-owned pools rather than a general heap.



Dynamic kernel object lifecycle

14.1 Configure bounded pools

Enable runtime creation of non-task objects in `kconfig.h` and set the maximum count for each required family:

```
#define RK_CONF_DYNAMIC_OBJECTS          ON
#define RK_CONF_DYNAMIC_SEMAPHORES_MAX  4U
#define RK_CONF_DYNAMIC_MUTEXES_MAX     4U
#define RK_CONF_DYNAMIC_SLEEP_QUEUES_MAX 2U
#define RK_CONF_DYNAMIC_MSG_QUEUES_MAX   4U
#define RK_CONF_DYNAMIC_TIMERS_MAX       2U
#define RK_CONF_DYNAMIC_MRMS_MAX         1U
```

`RK_CONF_DYNAMIC_OBJECTS` defaults to `ON` independently of `RK_CONF_DYNAMIC_TASK`. Define it as `OFF` to remove dynamic service objects. Dynamic objects and tasks use separate pools and may be configured independently.

Each maximum reserves a kernel-owned Memory Partition for that control-object type. A maximum of zero keeps that family unavailable even when dynamic objects are enabled. Exhausting a family returns `RK_ERR_BUFFER_EMPTY`; it does not fall back to a heap or wait for another object to be destroyed.

Initialise the partitions once before the first create operation:

```
VOID kApplicationInit(VOID)
{
    RK_INIT_OBJ_PARTITIONS

    /* Create startup tasks and static objects. */
}
```

The statement-style macro is a no-op when the feature is disabled. It intentionally discards the initialisation result. Startup code that must inspect failure may call `kObjPartitionsInit()` directly inside an `RK_CONF_DYNAMIC_OBJECTS` feature guard.

14.2 Static and dynamic forms

Property	Static object	Dynamic object
Control-block storage	declared by the application	allocated from a typed kernel pool
Start of lifetime	matching Init call	matching Create call
Reference	address of the object	typed handle such as <code>RK_MUTEX_HANDLE</code>
End of lifetime	normally application lifetime	matching Destroy call after quiescence
Service operations	unchanged	unchanged; pass the handle as the object pointer

Create allocates the kernel control block, not every buffer referenced by that control block:

Dynamic family	Storage that remains application-owned
Semaphore, Mutex, Sleep Queue	none
Message Queue or Mailbox	message buffer supplied to <code>kMsgQueueCreate()</code> or <code>kMboxCreate()</code>
Timer	callback argument and anything it references
MRM	<code>RK_MRM_BUF</code> array and payload pool supplied to <code>kMRMCreate()</code>

Application-owned storage must remain valid until the corresponding **Destroy** operation succeeds.

For example, a dynamically allocated Mutex is used by the ordinary Mutex operations:

```
static RK_MUTEX_HANDLE stateLock = NULL;

RK_ERR err = kMutexCreate(&stateLock,
                        RK_PRIO_INHERITANCE);
K_ASSERT((err == RK_ERR_SUCCESS) && (stateLock != NULL));

err = kMutexLock(stateLock, RK_WAIT_FOREVER);
K_ASSERT(err == RK_ERR_SUCCESS);
UpdateSharedState();
err = kMutexUnlock(stateLock);
K_ASSERT(err == RK_ERR_SUCCESS);

err = kMutexDestroy(&stateLock);
K_ASSERT((err == RK_ERR_SUCCESS) && (stateLock == NULL));
```

Create and Destroy take the address of the typed handle variable. A failed Create leaves that output handle `NULL`. A successful Destroy clears the handle variable passed to it and returns the control block to its family pool.

For a Message Queue, only the control block is dynamic:

```

#define WORK_DEPTH 4U

typedef struct
{
    ULONG command;
    ULONG argument;
} WorkItem;

static RK_MSG_QUEUE_HANDLE workQueue = NULL;
RK_DECLARE_MSG_QUEUE_BUF(workStorage, WorkItem, WORK_DEPTH)

RK_ERR err = kMsgQueueCreate(&workQueue,
                             workStorage,
                             RK_MSGQ_MSG_SIZE(WorkItem),
                             WORK_DEPTH);
K_ASSERT((err == RK_ERR_SUCCESS) && (workQueue != NULL));

/* Use workQueue with the ordinary Message Queue operations. */

/* First stop users and drain the queue. */
err = kMsgQueueDestroy(&workQueue);
K_ASSERT((err == RK_ERR_SUCCESS) && (workQueue == NULL));

```

`workStorage` is not released by `Destroy`. It is application storage and may be reused only after the queue has been destroyed successfully.

14.3 Destruction requires quiescence

`Destroy` is not a close or broadcast-cancellation operation. The application must first stop new users and allow current operations to finish.

Dynamic family	Required state before <code>Destroy</code>
Semaphore	no waiting tasks; a stored count may be discarded
Mutex	unlocked, no owner and no waiting tasks
Sleep Queue	no waiting tasks
Message Queue or Mailbox	empty, with no waiting senders, receivers or active broadcast receivers
Timer	may be armed; <code>Destroy</code> cancels it before release
MRM	no outstanding reader reference and no reserved unpublished buffer

The following rules apply to every dynamic family:

- `Create` and `Destroy` are task-context operations and must not be called from an ISR.
- A `Destroy` function accepts only an object produced by the matching `Create` function. Do not pass the address of a static object.
- The kernel clears only the handle variable passed to `Destroy`. Copied handles and raw pointer aliases are not tracked; after successful destruction they are stale.
- Quiescence is an application protocol. Prevent another task from starting an operation while the object is being destroyed.
- Give one component authority over the owning handle and the `Create/Destroy` operations. Other tasks may borrow the handle only while that component guarantees the object's lifetime.
- Application-owned supporting storage must outlive the control object: Message Queue buffers, MRM control and payload pools, and Timer callback arguments remain valid through successful destruction.

- Do not destroy a Timer from its own callback. The PostProc timer path still uses the Timer state after the callback returns.

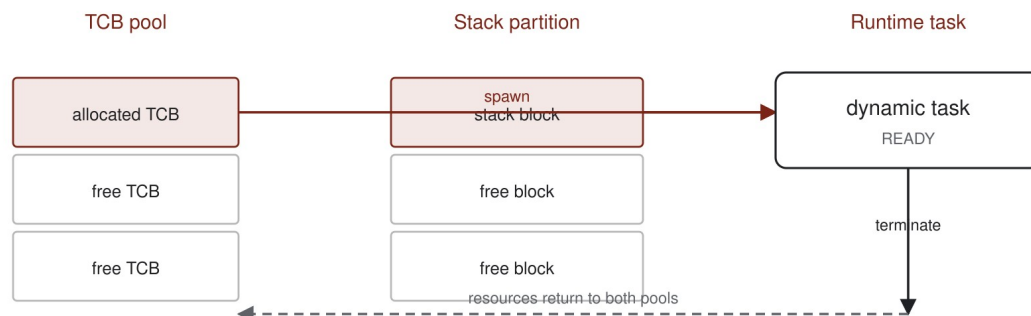
There are no dynamic-object APIs for Memory Partitions or Task Event Registers. Synchronous/Invocation and Asynchronous Direct Message endpoints remain task-backed. Direct-message pools remain application-owned Memory Partitions.

Trace registers a created object normally and unregisters it after successful destruction, so it no longer appears in object listings.

15 Dynamic tasks

Dynamic tasks support middleware that must create and terminate tasks after scheduling has started. They should still come from a bounded, known configuration. In RK0 0.71.0, `RK_CONF_DYNAMIC_TASK` is disabled by default and must be enabled deliberately.

```
#define RK_CONF_DYNAMIC_TASK ON
```



Dynamic task resource lifecycle

Each dynamic task uses:

- one TCB slot from the fixed kernel task pool;
- one stack block from an application-selected Memory Partition.

```
#define DYNAMIC_STACK_WORDS 256U
#define DYNAMIC_TASKS      2U

RK_DECLARE_DYNAMIC_TASK(workerHandle, WorkerTask)
RK_DECLARE_DYNAMIC_STACK_POOL(workerStackMemory,
                               workerStackPool,
                               DYNAMIC_TASKS,
                               DYNAMIC_STACK_WORDS)

static RK_DYNAMIC_TASK_ATTR workerAttributes = {
    .taskFunc = WorkerTask,
    .argsPtr = RK_NO_ARGS,
    .taskName = "DynWork",
    .priority = 3U,
    .preempt = RK_PREEMPT,
    .stackMemPtr = &workerStackMemory
};
```

Initialise the stack partition before spawning:


```
kMemPartitionInit(&workerStackMemory,
                  workerStackPool,
                  sizeof(workerStackPool[0]),
                  DYNAMIC_TASKS);

kTaskSpawn(&workerAttributes, &workerHandle);
```

A successfully spawned task becomes READY immediately and may preempt its creator.

Only runtime-spawned tasks may be terminated. `kTaskTerminate(&handle)` terminates another eligible dynamic task and sets that handle variable to `NULL`. `kTaskTerminateSelf()` defers cleanup to `PostProc` because the running task cannot reclaim its own active stack.

Termination is rejected while the task owns or waits for a resource, remains in an unsupported blocking or has dependants through Synchronous/Invocation or Asynchronous Direct Message endpoint state.

application's responsibility even after the handle passed to terminate is cleared.

Real-time rule: runtime creation changes the active task set. Bound when and how many tasks may exist. A dynamic task should use `kSleepUntil()` with a local anchor for periodic work; it did not participate in the scheduler-start phase used by `kSleepRelease()`.

16 Trace and fault handling

16.1 Trace console

When `RK_CONF_TRACE` is enabled, `kTraceInit()` starts an interactive UART-backed trace task. The board supplies a non-blocking receive hook and signals input from its UART ISR.

Name initialised objects so console output is readable:

```
kTraceNameObject(&sampleQueue, "SampleQ");
kTraceNameObject(&stateLock, "StateLk");
kTraceInit();
```

Names are limited by `RK_NAME_SIZE`; with the default size, use at most seven visible characters plus the terminating NUL.

Useful commands include:

Command	Use
top	Task state, effective and nominal priority, dispatches, CPU accounting and stack watermark
list kobjects	Registered kernel objects and last operation
list kmesg	Queue depth and waiting senders or receivers
list kipc	Task-backed Synchronous/Invocation and Asynchronous Direct Message state
list ksema	Semaphore and Mutex state
list kmem	Partition size and free blocks
list ktimers	Application timers
hist <name>	Recent operations for one object
hist task/<name>	Effective-priority changes for one task

Trace CPU percentage is diagnostic accounting, not a cycle-accurate profiler. Use the history with the task and object snapshots to understand causality.

16.2 Error handling

In checked builds, invalid API use can call `kErrorHandler()` in addition to returning an error. Configure the handler for the application's fault policy: halt, capture evidence, reset or hand control to a safety mechanism as appropriate.

Treat these as programming errors rather than routine branch conditions:

- using an uninitialised or wrong object;
- using a stale handle after dynamic object destruction;
- destroying a dynamic object while another task can still use it;
- blocking from an ISR;
- recursive Mutex lock;
- unlocking a Mutex without ownership;
- using Synchronous/Invocation operations while owning a Mutex;
- freeing a block to the wrong partition;
- letting a task return from its entry function.

Stack overflow and exhausted bounded resources should produce evidence that is useful after reset. The trace stack watermark, object history and application-specific retained fault record are complementary.

17 Choosing a communication service

The services differ mainly in what they retain and where completion occurs.

	Queue/Mbox	Synchronous	Invocation	Async Direct	MRM
Retains	bounded/one slot	none	none	endpoint queue	latest only
Success	copied/admitted	receiver copied	server replied	endpoint owns	published
Receiver	any waiter	named task	server task	named task	many readers
Priority relation	none	handoff	call duration	none	none

Comparison of RK0 message-passing services

Need	Choose	Success means
Count occurrences or units	Semaphore	one token was consumed or posted
Private bitwise occurrences	Task Event Register	requested bits were present and consumed
Wait for a future notification	Sleep Queue	the task was signalled after it began waiting

17.1 Completion and retention matrix

Need	Choose	Success means
Preserve a bounded message history	Message Queue	message copied to a receiver or queue storage
Keep only one pending newest value	overwrite Mailbox	one-slot value replaced or delivered
Deliver one value to present waiters	broadcast Mailbox	each waiting receiver obtained a copy
Transfer a pool-backed message	Asynchronous Direct Message	receiver endpoint owns the RK_MESG
Deliver directly and wait until copied	Synchronous Message	receiver copied the offered payload
Invoke a server and receive a result	Invocation	server replied
Publish latest state to many readers	MRM	complete value became current

17.2 Selection questions

Ask these questions in order:

1. Is this an occurrence, shared-state ownership, or a message?
2. If it is a message, should history be preserved, discarded, or never created?
3. Does the sender complete on admission, on copy, or on server reply?
4. Is there one owner, one receiver, one server, or many readers?
5. What priority dependency appears when a more urgent task waits?

Do not emulate a stronger contract with a weaker service unless the application explicitly accepts the changed completion, timeout, ownership, and scheduling semantics.

18 Application helper macros

kapi.h exposes compile-time helpers for aligned storage, sizing, common task access, and time conversion. Declaration macros reserve application storage; Init, Create, or Spawn establishes the service object or task.

18.1 Size calculations

Macro	Result	Principal use
RK_WORD_SIZE	bytes in one ULONG	common sizing unit
RK_TYPE_WORD_COUNT(Type)	sizeof(Type), rounded to words	Memory Partition blocks
RK_TYPE_SIZE_POW2_WORDS(Type)	1, 2, 4, 8, or 16 words	lower-level slot sizing
RK_MESGQ_MESG_SIZE(Type)	Queue slot width in words	Queue/Mailbox init
RK_MESGQ_BUF_SIZE(Type, N)	total Queue backing words	explicit Queue arrays
RK_MESG_BLOCK_SIZE_BYTES(Type)	RK_MESG header plus payload, word-aligned	direct-message pool block
RK_MESG_POOL_WORDS(Type, N)	total direct-message pool words	direct-message pool arrays

Message Queue payloads are limited to 1, 2, 4, or 8 words. A larger type may round to 16 words, but queue initialisation rejects it. Direct-message pool helpers include the RK_MESG header automatically and permit an application-defined payload type.

```
RK_DECLARE_MEM_POOL(LogRecord, logPool, LOG_BLOCKS)
kMemPartitionInit(&logMemory, logPool,
    sizeof(LogRecord), LOG_BLOCKS);

RK_DECLARE_MESG_POOL(asyncPool, asyncBuf, AsyncPayload_t, 4U)
kMesgPoolInit(&asyncPool, asyncBuf, sizeof(AsyncPayload_t), 4U);
```

RK_TYPE_WORD_COUNT() returns words; sizeof() returns bytes. kMesgPoolInit() receives payload bytes, while RK_MESG_POOL_WORDS() sizes the complete backing array.

18.2 Declaration and initialisation helpers

Macro	Declares or performs	Follow with
RK_DECLARE_TASK	entry prototype, aligned static stack, handle	kTaskInit()/kCreateTask()
RK_DECLARE_DYNAMIC_TASK	entry prototype and handle	kTaskSpawn()
RK_DECLARE_DYNAMIC_STACK_POOL	stack Memory Partition and blocks	kMemPartitionInit()
RK_DECLARE_MESG_QUEUE_BUF / RK_DECLARE_MESG_QUEUE	Queue storage; optionally control block	kMesgQueueInit()
RK_DECLARE_MBOX_BUF / RK_DECLARE_MBOX	one-slot Queue storage; optionally Mbox	kMboxInit()
RK_DECLARE_MESG_POOL_BUF / RK_DECLARE_MESG_POOL	direct-message blocks; optionally partition	kMesgPoolInit()
RK_DECLARE_MEM_POOL	word-aligned fixed blocks	kMemPartitionInit()
RK_INIT_OBJ_PARTITIONS	bounded dynamic control-block pools	once before Create calls

Queue and Mailbox helpers encode payload type and capacity only into the C array size; the kernel does not retain that C type. Pass matching type-derived size and capacity to initialisation.

```
typedef struct { UINT command; ULONG value; } Request_t;

RK_DECLARE_MESG_QUEUE(requestQueue, requestStorage, Request_t, 4U)
kMesgQueueInit(&requestQueue, requestStorage,
               RK_MESGQ_MESG_SIZE(Request_t), 4U);

RK_DECLARE_MESG_POOL(requestPool, requestPoolStorage, Request_t, 4U)
kMesgPoolInit(&requestPool, requestPoolStorage,
              sizeof(Request_t), 4U);
```

Use a Queue when the kernel copies values into bounded slots. Use a direct-message pool when a task-addressed endpoint should take ownership of the allocated message object.

All declaration macros reserve application storage. Their lifetime must cover every task and kernel path that can still reference that storage.

18.3 Time and task lookup helpers

Macro	Meaning	Important constraint
<code>RK_MS_TO_TICKS</code> (ms)	Milliseconds divided by the configured tick interval	Truncates; a sub-tick value becomes zero
<code>RK_TICKS_TO_MS</code> (ticks)	Ticks multiplied by the configured tick interval	Does not saturate on overflow
<code>RK_RUNNING_PID</code>	PID of the running task	Use only after scheduling has started
<code>RK_RUNNING_PRIORITY</code>	Current effective priority, including inheritance or boost	Not necessarily the assigned priority
<code>RK_RUNNING_NOMINAL_PRIORITY</code>	Assigned nominal priority	Does not show a temporary priority change
<code>RK_RUNNING_HANDLE</code>	Handle of the running task	Task context after scheduler start
<code>RK_RUNNING_NAME</code>	Pointer to the running task's name	Pointer remains owned by the task object
<code>RK_TASK_PID(HANDLE)</code>	PID of the selected task	Handle must be valid
<code>RK_TASK_PRIORITY(HANDLE)</code>	Effective priority of the selected task	May reflect inheritance or boost
<code>RK_TASKNAME_PTR(HANDLE)</code>	Pointer to the selected task's name	Directly dereferences the handle; do not use a stale or null handle

The conversion helpers express arithmetic, not delay policy. After `RK_MS_TO_TICKS(1U)` returns zero on a coarse tick configuration, a timeout-bearing service observes `RK_NO_WAIT`; it does not round the request up to one tick.

19 Compact API index

This index is for orientation. See `kapi.h` for exact parameters, conditional compilation, and return values.

Family	Principal operations
Kernel and tasks	<code>kInit</code> , <code>kTaskInit</code> / <code>kCreateTask</code> , <code>kYield</code> , task accessors
Dynamic objects/tasks	<code>RK_INIT_OBJ_PARTITIONS</code> , <code>kObjPartitionsInit</code> , <code>kTaskSpawn</code> , <code>kTaskTerminate</code>
Scheduler and time	<code>kSchLock</code> / <code>kSchUnlock</code> , <code>kSleepDelay</code> , <code>kSleepRelease</code> , <code>kSleepUntil</code> , <code>kTickGet</code>
Application Timer	<code>kTimerInit</code> / <code>kTimerCreate</code> , <code>kTimerCancel</code> , <code>kTimerDestroy</code>
Task Event Register	<code>kEventSet</code> , <code>kEventGet</code> , <code>kEventQuery</code> , <code>kEventClear</code>
Semaphore	<code>kSemaphoreInit</code> / <code>Create</code> , <code>kSemaphorePend</code> , <code>kSemaphorePost</code> , <code>Query</code> , <code>Destroy</code>
Sleep Queue	<code>kSleepQueueInit</code> / <code>Create</code> , <code>Sleep</code> , <code>Signal</code> , <code>Wake</code> , <code>Ready</code> , <code>Unready</code> , <code>Query</code> , <code>Destroy</code>
Mutex	<code>kMutexInit</code> / <code>Create</code> , <code>kMutexLock</code> , <code>kMutexUnlock</code> , <code>kMutexQuery</code> , <code>kMutexDestroy</code>
Condition Variable	<code>kCondVarInit</code> , <code>kCondVarWait</code> , <code>kCondVarSignal</code> , <code>kCondVarBroadcast</code>
Message Queue	<code>kMesgQueueInit</code> / <code>Create</code> , <code>Send</code> , <code>Recv</code> , <code>Jam</code> , <code>Peek</code> , <code>Reset</code> , <code>Query</code> , <code>Destroy</code>
Mailbox	<code>kMboxInit</code> / <code>Create</code> , <code>Post</code> , <code>Pend</code> , <code>overwrite and broadcast operations</code> , <code>Destroy</code>
Synchronous Message	<code>kSynchMesgInit</code> , <code>kSynchMesgSend</code> / <code>kSynchSendWait</code> , <code>kSynchMesgRecv</code> / <code>kSynchRecv</code>
Invocation	<code>kSynchMesgCall</code> / <code>kSynchMesgInvoke</code> , <code>kSynchMesgAccept</code> , <code>kSynchMesgReply</code>
Async Direct Message	<code>kMesgEndpointInit</code> , <code>kMesgPoolInit</code> , <code>kMesgAlloc</code> , <code>kMesgSend</code> , <code>kMesgWait</code> , <code>kMesgFree</code>
MRM	<code>kMRMInit</code> / <code>Create</code> , <code>kMRMReserve</code> , <code>kMRMPublish</code> , <code>kMRMGet</code> , <code>kMRMUnget</code> , <code>Destroy</code>
Memory Partition	<code>kMemPartitionInit</code> , <code>kMemPartitionAlloc</code> , <code>kMemPartitionFree</code>
Trace	<code>kTraceInit</code> , <code>kTraceNameObject</code> , <code>snapshot and history APIs</code>

20 Further reading

- [RK0 source and public API](#)
- [RK0 change log](#)
- [RK0 Service Map](#)
- [RK0 DocBook](#)
- [Application structure and examples](#)